Velo — CIO Call Preparation

Likely questions and suggested answers · Internal prep note, not for distribution · 28 Oct 2026

> **Before the call: three things to settle**
>
> 1. **Can you name Indian Terrain as a reference?** Check with them first. If not, say "a 160-store menswear retailer".
> 2. **Be honest about status.** Velo is in a parallel run. It has not yet sent live orders into an ERP. Say so before he asks.
> 3. **Don't quote ROI figures.** "We'll measure it against your own baseline during the parallel run" is the safer answer.

1. Product and value

Q: What does Velo actually do?

It runs merchandise allocation in two parts:

- **Pre-season Initial Fill:** splits the buy plan across stores using a demand model trained on each store's own sales history. It never puts more stock on a shelf than the store's planogram allows.
- **Daily in-season replenishment:** every night it decides, per store and style, whether to refill from the warehouse, transfer from another store, replace with a visually similar style, or pull back stock that has stopped selling.

Q: How is this different from our ERP's replenishment or a tool like Blue Yonder?

- ERP replenishment is rule-based min/max at SKU level.
- Velo works at the level a fashion planner thinks in: options (style + colour), size curves and fashion buckets.
- It handles fashion-specific cases ERP doesn't: core vs non-core sizes, swapping a sold-out option for a lookalike, and moving stock between stores before buying more.
- Compared with the large suites, it is lighter, fashion-specific, and runs inside the client's own data platform.

Q: Does it replace my planners?

No. Same-SKU refills from the warehouse are automatic. Replacements, pull-backs and store transfers need a planner's approval. Planner overrides are capped, and any exception needs Head of Planning sign-off. Every decision is logged.

Q: Can planners see why the system recommended something?

Yes. Every allocation line carries a plain-language reason (e.g. "expected to sell 2 here, capped by planogram"), and the screens show the buyer's plan and the system's plan side by side.

2. Architecture and data

Q: What's the technology stack?

- Databricks Lakehouse on Azure, governed by Unity Catalog.
- Data processed in layers: raw → cleaned → analytical.
- Demand model: LightGBM. Image similarity for replacements: CLIP, searched through Databricks Vector Search.
- Planner screens: a web app on Databricks Apps.

Q: Where does our data live? Does it leave our tenant?

Everything is deployed into **your own** Azure subscription and Databricks workspace. Your data never goes to a Simbus environment, and all tables are open Delta format that you own.

Q: What data do you need from us?

- **Daily feeds:** POS sales, end-of-day store stock, warehouse inventory, store receipts, order and dispatch status from the ERP.
- **Masters:** product master; store master (grade, region, buddy store).
- **Season inputs:** buy plan, planogram, size depth, fashion bucket norms.
- Product images, if you want the replacement feature.

Q: How much history do you need?

Ideally two years of POS. Most other feeds work with recent data only. A new store with no history borrows a similar "buddy" store's history until it has its own.

Q: What if our data quality is poor?

Bad rows are not dropped silently: they are set aside in a quarantine table and counted. Be candid that **data readiness is the biggest delivery risk** (stale master data, store codes missing from the store master, overwritten season tags). Velo surfaces these problems, but the client has to fix them at source.

3. Integration

Q: How does Velo integrate with our ERP?

Through files, not live API calls:

- Feeds arrive as file drops (SFTP or upload to cloud storage).
- Approved orders go out as CSVs in the ERP's own upload format (sales orders and transfer orders).
- Loosely coupled: no ERP changes and no risk to ERP stability.

Q: Which ERPs do you support?

Today: Microsoft Dynamics AX formats. The order ID is already designed to fit SAP's field limits. Any ERP that accepts file uploads is a format-mapping exercise, not a rebuild.

Q: Real-time?

No — a daily overnight batch of about 30 minutes, which matches how replenishment actually happens. **Don't promise real-time.**

4. Security and governance

Q: Who can see and do what?

- Users sign in with Databricks single sign-on.
- Role-based access: Planner, Head of Planning, Admin.
- Each planner sees only the stores assigned to them.
- Every admin change and planner decision goes to an audit log.

Q: What access does Simbus need?

For setup, access to configure the Azure and Databricks resources. Who runs it day to day after go-live is still open — say "to be agreed". Don't commit Simbus to ongoing operations on the call.

Q: How is licensing enforced?

- A signed annual licence file, checked inside your environment with no call back to Simbus.
- If it lapses, the planning screens lock and order export stops; data ingestion keeps running, so no history is lost.
- **Don't call it tamper-proof.** Say "offline-verified signed licence".

5. Deployment, cost and ownership

Q: How long does implementation take?

Reference model: about 13 weeks of build, then a 13-week parallel run alongside current processes. Actual length depends mostly on data readiness.

Q: What will it cost to run, beyond the licence?

- Databricks and Azure usage is billed to your subscription.
- Scheduled job clusters, not always-on compute, keep usage low.
- Don't give a figure — offer to size it once you know store and SKU counts.

Q: Commercial model?

Annual licence plus support (AMS). Keep figures for the follow-up proposal.

Q: We're on AWS or GCP, or we don't use Databricks.

Databricks runs on all three clouds, but Velo has only been deployed on Azure — say exactly that. A client without Databricks needs a workspace provisioned; Simbus handles that setup.

Q: What if we stop using Velo?

All data and outputs stay in your lakehouse as open Delta tables. No lock-in on the data.

6. AI and models

Q: Is the AI accurate? How do you know?

- Every training run tests the demand model against a held-out past season, and keeps it only if it beats a simple history-based rule.
- On backtests it outperformed history-based formulas.
- The real answer comes from the parallel run on your data.

Q: Any GenAI?

Yes, in a supporting role: an "ask the data" assistant (Genie) for natural-language questions, and AI-written explanations of why a replacement style is similar. Decisions themselves come from the forecasting and optimisation logic, not from an LLM.

Q: What about a new store or a new style with no history?

New stores borrow history from a similar buddy store. New styles are scored from category and store patterns, and matched to similar past styles visually.

7. Proof and maturity

Q: Who's using it? Results?

The first installation is a 160-store fashion retailer (name them only if cleared), currently in its parallel run. Be clear that results are being measured now, and offer a demo with real screens.

Q: How big can it scale?

Runs today on about 160 stores and about 366k barcodes on a small cluster. Databricks scales horizontally, so larger networks mean more compute, not a redesign.

8. Architecture and data engineering

For when the CIO moves from "what does it do" to "how is it built and run". Gaps are marked — be honest about them.

Data platform and design

- **Medallion layers:** raw (bronze) → cleaned (silver) → analytical (gold) → serving. Raw data is never modified, so anything downstream can be rebuilt from it.
- **Open format:** everything is Delta tables. Any Spark engine can read them, and Power BI and SQL tools can query them directly.
- **Idempotent runs:** every write is a MERGE on a fixed key, so re-running a job does not duplicate orders.
- **Schema changes in a feed:** new or unexpected columns are captured in a separate column instead of breaking the load. Bad rows go to a quarantine table and are counted.
- **Data contracts:** each feed has a template plus validation rules kept in a config file. Rules can change without code changes.

Governance and security

- **Unity Catalog:** central permission model, table-level grants, lineage and audit.
- **PII:** Velo needs no customer data. POS is used at store × product × day level. A strong answer — expect the question.
- **Secrets and identities:** CI/CD deploys through a service principal using OAuth. The planned upgrade removes stored secrets altogether (Workload Identity Federation).
- **Network:** private endpoints / VNet injection are not documented yet. Say they would follow the client's Azure standards and be confirmed during design.

Engineering practice

- **Infrastructure as code:** every job, pipeline, schema and the app is defined in Databricks Asset Bundles. Nothing is created by hand in the UI.
- **CI/CD:** Azure DevOps runs validate → deploy to dev → deploy to prod, with a manual approval gate before prod.
- **Environments:** separate dev and prod catalogs; a staging target exists in the bundle.
- **Testing:** unit tests run in CI. Be honest that test coverage is still growing.

Operations

- **Monitoring:** jobs alert on failure; data-quality counts and pipeline event logs are available. No formal SLA dashboard yet.
- **Recovery:** Delta time travel restores any table to an earlier version; storage redundancy comes from Azure (ADLS). Formal DR plan with RTO/RPO targets not yet defined — offer to agree one.
- **Run window:** data lands overnight, Gold finishes around 01:30, replenishment runs at 02:00. Late data is the main operational risk; jobs check that feeds are current and stop with a clear "data required" message rather than produce wrong orders.
- **Support model:** who runs it after go-live is still open.

Cost

- Scheduled job clusters, not always-on compute or serverless; the SQL warehouse auto-stops when idle.
- For "how do you stop costs creeping up?": cost tags and right-sized clusters. Don't claim cluster policies are enforced on every job — a check on 2026-08-28 found no job references one.

ML governance

- Models versioned in MLflow with a Production alias; promotion is a manual step after review.
- The size-curve model fitted at season start stays frozen for that season, so results don't shift mid-season.

Lock-in

- **Data:** none — open Delta in the client's own storage.
- **Platform:** Velo depends on Databricks. Say it plainly: a deliberate choice for governance, ML and scale in one platform.

Questions to ask him

- How many stores, styles and SKUs? Which ERP? Which cloud, and do you already use Databricks?
- Is your main pain the initial allocation, in-season stock-outs, or dead stock?
- How do planners work today: spreadsheets, an ERP module, or a suite?
- Who owns the data feeds internally, and can POS and stock come in daily?
- What would make a parallel run count as a success for you?
- What are your cloud and network standards: private networking, landing zone, security review process?
- Do you already have a data platform? Velo could read from your existing lakehouse instead of new file drops.
- Which reporting tool do you use (Power BI or other)? The gold tables can feed it directly.

Things to avoid saying

- "Real-time", "fully automated" or "tamper-proof".
- Any ROI or sell-through uplift figure.
- That Velo is live in production with ERP orders.
- That Simbus will run it day to day (not yet decided).
- That DR, private networking or cluster policies are already in place.